MODULE 1

ARDUINO BASICS

1.INTRODUCTION

The **Arduino Uno** is a widely used open-source microcontroller board based on the ATmega328P microcontroller. It provides a simple and versatile platform for building electronic projects, from basic sensor interfacing to advanced robotics and IoT applications. With 14 digital input/output pins, 6 analog inputs, and multiple communication interfaces such as UART, I2C, and SPI, the Arduino Uno allows easy connection with a variety of sensors, actuators, and modules. It can be powered via USB or an external power supply and is programmed using the user-friendly Arduino IDE, making it ideal for beginners, students, and hobbyists in electronics and embedded systems.

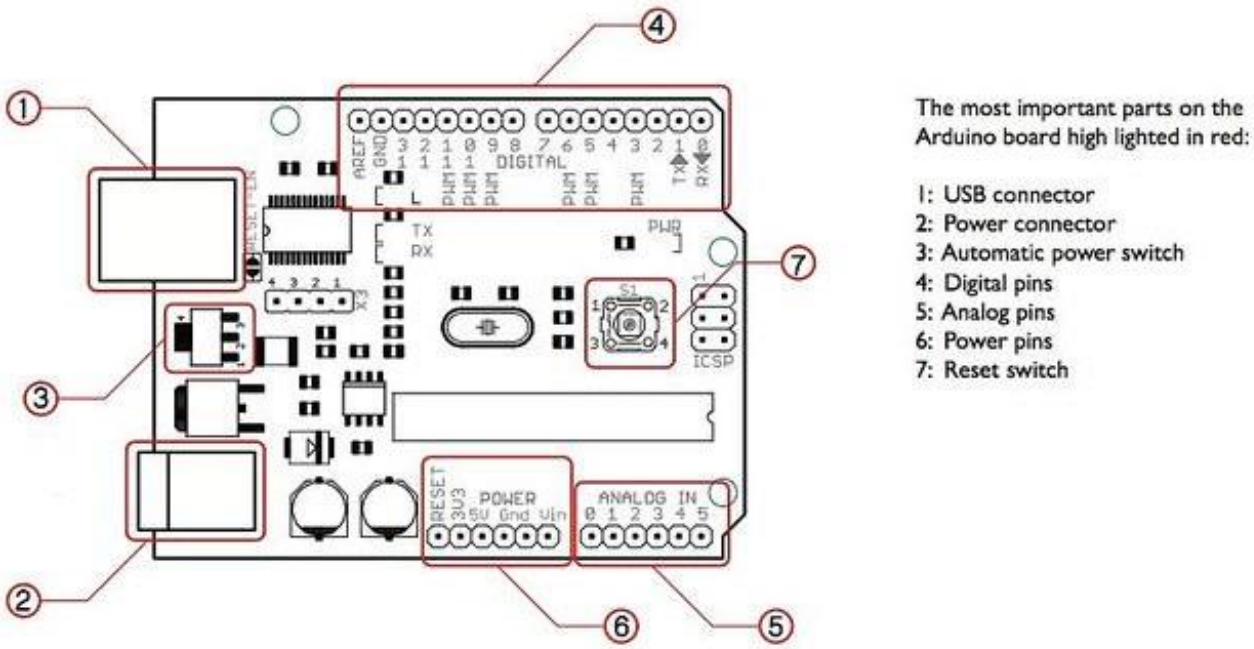
Advantages of Arduino Technology

- It is cheap
- It comes with an open supply hardware feature that permits users to develop their own kit
- The software of the Arduino is well-suited with all kinds of in operation systems like Linux, Windows, and Macintosh, etc.
- It also comes with open supply software system feature that permits tough software system developers to use the Arduino code to merge with the prevailing programing language libraries and may be extended and changed.
- For beginners, it is very simple to use.

1.1 ARDUINO UNO R3 COMPONENTS

The Arduino Uno R3 is a versatile microcontroller board that serves as an ideal platform for learning and developing electronic projects. At its core, it features the ATmega328P microcontroller, which provides 32 KB of flash memory, 2 KB of SRAM, and 1 KB of EEPROM, operating at a clock speed of 16 MHz. The board can be powered via a USB connection or an external DC supply through the power jack, with onboard voltage regulators supplying stable 5V and 3.3V outputs. It offers 14 digital input/output pins, including 6 that support PWM output, and 6 analog input pins for interfacing with sensors. Communication with other devices is supported

through UART, I2C, and SPI interfaces, while additional features such as the reset button, ICSP header, crystal oscillator, and onboard LEDs enhance its functionality and usability. The combination of these components makes the Arduino Uno R3 an accessible and powerful tool for prototyping, robotics, home automation, and IoT applications.



1. Microcontroller (ATmega328P)

   - 8-bit microcontroller
   - B Flash memory (0.5 KB used by bootloader)
   - 2 KB SRAM, 1 KB EEPROM
   - 16 MHz clock speed

2. Power Supply

   - **Power Jack**: Accepts 7–12V DC input
   - **USB Port**: 5V power supply and programming interface
   - **Voltage Regulators**: Provide stable 5V and 3.3V outputs

3. Digital I/O Pins

- **14 digital pins** (D0–D13)
- **PWM pins**: 6 pins (D3, D5, D6, D9, D10, D11)
- **Built-in LED**: Connected to pin 13

4. Analog Input Pins

- **6 Analog inputs** (A0–A5)
- Can read voltages from sensors and convert them to digital values

5. Communication Interfaces

- **I2C**: A4 (SDA), A5 (SCL)
- **SPI**: 10 (SS), 11 (MOSI), 12 (MISO), 13 (SCK)

6. Reset and Control

- **Reset Button**: Restarts the program
- **ICSP Header**: For programming the microcontroller directly

7. Additional Components

- **Crystal Oscillator**: 16 MHz for timing
- **LED Indicators**: Power LED, TX/RX LEDs for communication status
- **Micro USB Connector** (for programming and power)

1.2 MICROCONTROLLERS IN OTHER ARDUINOS

All the original Arduino boards, including the Arduino Uno, used an 8-bit Atmel, ATmega microcontroller based on the AVR architecture (Advanced Virtual RISC). Arduino has been expanding the range of available Arduino boards to include some that run on Intel (x86 and ARC—Argonaut RISC Core) architectures, and some that use ARM (Advanced RISC Machine) architectures.

The Arduino Due, for example, uses a 32-bit Arm Cortex-M3 microprocessor. This Cortex processor is faster and contains more peripherals than the 8-bit AVR MCU, thus enabling the Due to do things like play music. Other new Arduino boards add functionality like built-in Wi-Fi and Bluetooth, which is also facilitated by faster and more capable processors

- AVR—A microcontroller architecture developed by Atmel for their ATmega MCUs.
- ARM—A collection of 32/64-bit processor architectures developed by a company of the same name. Arm licenses its embedded architecture designs to be used by companies like Microchip and others.
- Cortex-M Series—Cortex M0, M3, and so on represent microprocessor ARM architectures.

1.3 PROGRAMMING INTERFACES

The **Arduino Uno** supports multiple programming and communication interfaces that allow uploading code, debugging, and interacting with external devices. The main programming interfaces are:

1. USB Interface

   The USB interface is the primary method for uploading programs to the Arduino Uno. Through its USB Type-B port, the board connects to a computer and transfers code using the Arduino IDE with the help of the onboard ATmega16U2 USB-to-serial converter. This interface also provides a 5V power supply, making both programming and powering the board easy and convenient.
2. UART (Serial Communication)

   The Arduino Uno supports UART serial communication through pins 0 (RX) and 1 (TX), allowing the microcontroller to communicate with computers, Bluetooth modules, and other serial devices. It is also used by the Serial Monitor for debugging. Since the USB interface internally relies on UART, it plays an important role in both programming and data communication.
3. ISP (In-System Programming) Header

   The In-System Programming (ISP) header is a 6-pin connector that enables direct programming of the ATmega328P without using the bootloader or USB interface. It is commonly used to burn a new bootloader or upload firmware using AVR programmers like USBasp. This interface is useful for advanced users who need low-level access or want to recover a corrupted microcontroller
4. Bootloader Interface

   The Arduino Uno includes a preinstalled bootloader that allows programs to be uploaded through the USB connection without extra hardware. It works through the UART interface

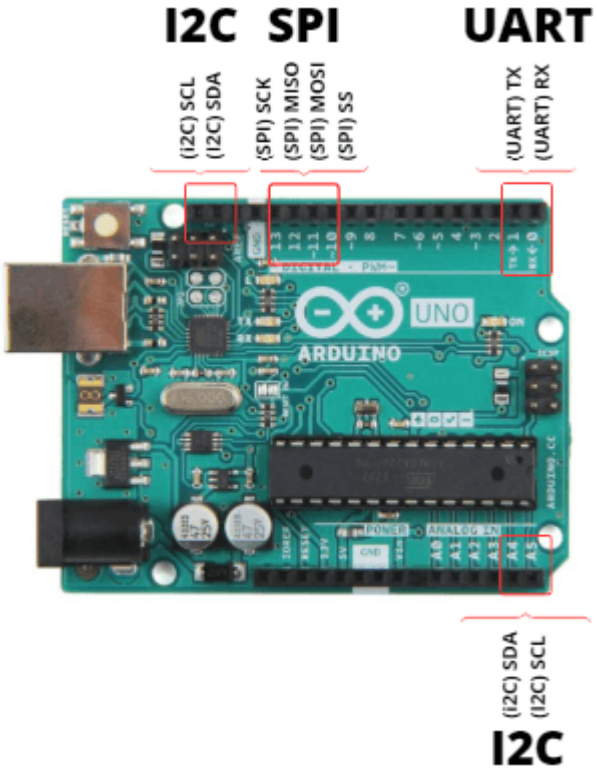
and simplifies the programming process, making the board beginner-friendly and allowing quick uploading and testing of sketches..

5. Software Programming Interface (Arduino IDE / PlatformIO)

   Programming the Arduino Uno is simplified through tools like the Arduino IDE, Web Editor, and PlatformIO, which use an easy-to-learn version of C/C++ with many built-in libraries. These platforms make it simple to work with sensors, motors, and displays, and their user-friendly environment ensures quick and straightforward code uploading.

6. Communication Interfaces Used in Programs (I2C and SPI)

   In addition to programming interfaces, the Arduino Uno includes I2C and SPI communication interfaces for connecting external modules. I2C uses pins A4 and A5 for multi-device communication, while SPI uses pins 10–13 for high-speed data exchange with devices like SD cards and displays



1.4 DIGITAL INPUTS, OUTPUTS AND PWM

Digital outputs on the Arduino Uno allow the microcontroller to send signals to external components such as LEDs, buzzers, relays, and motors. When a pin is configured as a digital output, it can produce only two possible states: **HIGH (5V)** or **LOW (0V)**. These states are controlled in a program using the digitalWrite( ) function after setting the pin mode to OUTPUT using pinMode( ). A HIGH output can turn on an LED or activate a module, while a LOW output turns it off. Digital outputs are essential for controlling external devices and creating interaction in embedded systems.

PWM (Pulse Width Modulation) is a technique used by the Arduino Uno to generate analog-like output using digital pins. Instead of producing a continuous analog voltage, PWM rapidly switches a pin between HIGH and LOW at a fixed frequency while varying the time spent in the HIGH state (duty cycle). A higher duty cycle means more power delivered to the device. On the Arduino Uno, PWM is available on specific pins marked with a "~" symbol (3, 5, 6, 9, 10, and 11). It is commonly used for controlling LED brightness, motor speed, and other devices that require variable output levels. The analogWrite( ) function is used to generate PWM signals in Arduino programs.

1.5 CONTROLLING AN LED ON A BREADBOARD WITH digitalWrite( )

Turning ON & OF an LED

```
const int LED=9;  // Define LED for pin 9
void setup()
{
pinMode (LED, OUTPUT); // Set the LED pin as an output
}
void loop()
{
digitalWrite(LED, HIGH); // Set the LED pin high
delay(1000);
digitalWrite(LED, LOW); // Set the LED pin Low
delay(1000);

}
```

LED WITH CHANGING BLINK RATE

```
const int LED=9; // Define LED for Pin 9
```

```
void setup()
{
pinMode (LED, OUTPUT); // Set the LED pin as an output
}
void loop()
{
for (int i=100; i<=1000; i=i+100)
{
digitalWrite(LED, HIGH);
delay(i);
digitalWrite(LED, LOW);
delay(i);
}
}
```

LED WITH CHANGING BLINK RATE

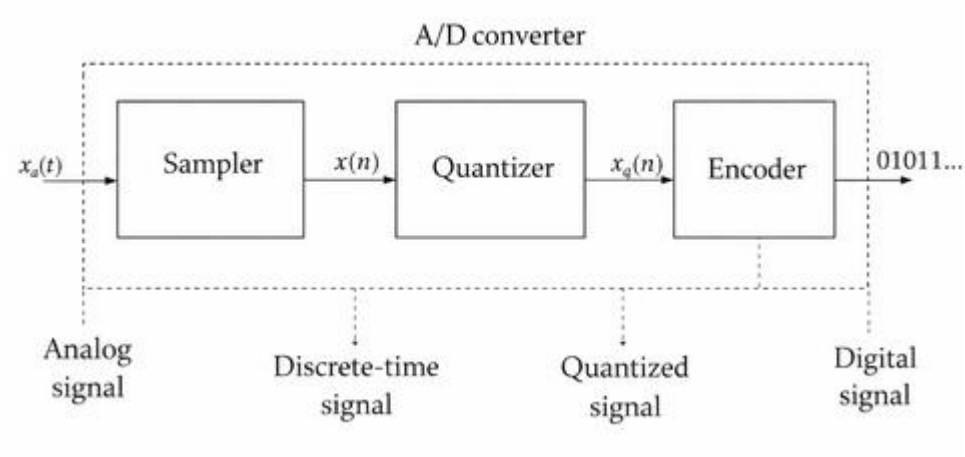
```
const int LED=9; // Define LED for Pin 9
void setup()
{
pinMode (LED, OUTPUT); // Set the LED pin as an output
}
void loop()
{
for (int i=100; i<=1000; i=i+100)
{
digitalWrite(LED, HIGH);
delay(i);
digitalWrite(LED, LOW);
delay(i);
}
}
```

1.6 INTERFACING ANALOG SENSORS

Analog and digital signals are the two basic types of signals used in electronic systems. An **analog signal** is a continuous signal that can have any value within a given range and varies smoothly with time. Examples include sound waves, temperature signals, and voltage from a microphone. Analog signals are highly sensitive to noise and interference, which can distort the signal during transmission or processing. A **digital signal**, on the other hand, has discrete values

and usually represents information in the form of binary digits, that is logic '0' and logic '1'. Digital signals are widely used in microcontrollers, computers, and digital communication systems because they are less affected by noise, easier to store and process, and more reliable for long-distance transmission. Most modern electronic systems convert analog signals into digital form for accurate processing and control.

Converting an analog signal to a digital signal is an essential process in digital systems such as microcontrollers and computers. This conversion is performed using an **Analog-to-Digital Converter (ADC)**. The process involves three main steps: **sampling, quantization, and encoding**. In sampling, the continuous analog signal is measured at regular time intervals. In quantization, each sampled value is approximated to the nearest discrete level. In encoding, the quantized value is converted into a binary number that can be processed by digital circuits. The accuracy of conversion depends on the resolution of the ADC, which is determined by the number of bits used. Analog-to-digital conversion allows real-world signals like temperature, light, and pressure to be processed digitally for monitoring and control applications.



1.7 ADC RESOLUTION AND QUANTIZATION

ADC resolution refers to the **smallest change in analog input voltage** that can be detected and represented by an Analog-to-Digital Converter. It depends on the **number of bits** used by the ADC. An *n-bit ADC* can represent 2^n discrete digital levels over its input voltage range. Higher resolution means more levels and hence **better accuracy** in representing the analog signal. The resolution (step size) of an ADC is given by

$$Resolution = \frac{V_{ref}}{2^n}$$

where V_{ref} is the reference voltage. For example, a 10-bit ADC with a 5 V reference has a resolution of approximately 4.88 mV per step. Higher ADC resolution reduces conversion error and is essential in applications requiring precise measurements such as temperature sensing, biomedical instrumentation, and data acquisition systems.

Quantization is the process of converting the **continuous amplitude values** of a sampled analog signal into a **finite number of discrete levels**. After sampling, each sample value is rounded to the nearest quantization level, and this process is known as quantization. The difference between the actual analog value and the quantized value is called **quantization error**, which is unavoidable in digital systems. The number of quantization levels depends on the resolution of the ADC; higher resolution results in smaller step size and lower quantization error. Quantization enables analog signals to be represented in digital form so that they can be processed, stored, and transmitted efficiently by digital systems such as microcontrollers and computers.

Pblm) An 8-bit Analog-to-Digital Converter (ADC) has a reference voltage of 5 V. Find the resolution (step size) of the ADC.

The resolution of an ADC is given by the formula:

$$Resolution = \frac{V_{ref}}{2^n}$$

Where:

- Vref=5 V
- n=8 bits

$$Resolution = \frac{5}{2^8} = 0.01953V = 19.53mV$$

QUESTION & ANSWERS

1. Design the value of resistance required to connect an LED to the digital output pin of an Arduino

When an LED is connected directly to the digital output pin of an Arduino, it must be protected from excessive current. LEDs are current-driven devices, and if excessive current flows through them, they may get permanently damaged. Therefore, a current-limiting resistor is connected in series with the LED to ensure safe operation.
The Arduino Uno digital output pin provides a voltage of approximately **5 V** when it is in the HIGH state. A typical LED has a forward voltage drop of about **2 V** for red LEDs (around 3–3.3 V for blue or white LEDs). The recommended forward current for standard LEDs is about **20 mA (0.02 A)**, though safe operation can also be achieved at lower currents.
Using **Ohm's law**, the value of the current-limiting resistor can be calculated as

$$R = \frac{V_{Source} - V_{LED}}{I_{LED}}$$

Substituting the values for a red LED,

$$R = \frac{5V - 2V}{0.02A}$$

$$R = \frac{3}{0.02A}$$

$$R = 150\Omega$$

Thus, the calculated resistance value is **150 Ω**. However, in practical circuits, the next higher standard resistor value is selected to ensure additional safety. Therefore, a **220 Ω resistor** is commonly used with Arduino digital pins. This limits the LED current to a safe value while still providing sufficient brightness.

2. What is the function of the bootloader in Arduino ?

The bootloader in Arduino is a small program that is pre-stored in the flash memory of the microcontroller. Its main function is to allow the Arduino board to be programmed directly from a computer without the need for an external hardware programmer.

When the Arduino board is powered ON or reset, the bootloader runs first. It checks whether a new program is being sent from the computer through the USB interface. If a new program is detected, the bootloader receives the program and writes it into the flash memory of the microcontroller.

If no new program is detected, the bootloader transfers control to the already stored user program and allows it to execute normally. The bootloader also handles serial communication between the Arduino and the computer during program uploading.

3. What is the difference between map( ) and constrain ( ) functions? Mention typical use of these functions.

The map() and constrain() functions in Arduino are commonly used for scaling and limiting values, especially while working with sensors and actuators. However, they serve different purposes.

The map() function is used to **convert a value from one range to another range**. It linearly scales an input value from its original range to a new desired range. For example, a sensor value read using analogRead() ranges from 0 to 1023, but this value may need to be converted to a PWM range of 0 to 255 for controlling LED brightness or motor speed.

The syntax of the map() function is:
map(value, fromLow, fromHigh, toLow, toHigh)

A typical use of map() is converting analog sensor readings into meaningful control values, such as mapping a potentiometer value to servo angle or LED brightness.

The constrain() function is used to **limit a value within a specified minimum and maximum range**. If the value exceeds the given range, it is forced to stay at the nearest limit. Unlike map(), it does not scale values; it only restricts them.

The syntax of the constrain() function is:
constrain(value, min, max)

A typical use of constrain() is protecting hardware by ensuring that a variable does not exceed safe operating limits, such as preventing PWM values from going beyond 0 to 255 or restricting sensor values within expected boundaries.

4. Explain the principle behind the analogwrite() function. Write code to vary the brightness of an LED using analogwrite()

The analogWrite() function in Arduino is used to generate a variable output that behaves like an analog signal using a digital pin. Since most Arduino boards do not have a true Digital-to-Analog Converter, analogWrite() works on the principle of **Pulse Width Modulation (PWM)**.

In Pulse Width Modulation, a digital output pin is switched ON and OFF at a fixed frequency. The amount of time the signal remains ON compared to the total time period is called the **duty cycle**. By changing the duty cycle, the average voltage delivered to the load is varied. A higher duty cycle results in a higher average voltage, and a lower duty cycle results in a lower average voltage.

The analogWrite() function accepts values from **0 to 255**, corresponding to duty cycles from **0% to 100%**. A value of 0 keeps the output always LOW, turning the LED OFF, while a value of 255 keeps the output always HIGH, making the LED glow at maximum brightness. Intermediate values produce proportional brightness levels. This rapid switching is not visible to the human eye, so the LED appears to glow steadily with varying brightness.

Thus, analogWrite() enables smooth control of LED brightness and motor speed using PWM, even though the output pin is digital.

<u>Program</u>

```
int ledPin = 9;    // PWM pin connected to LED

void setup()
 {
  pinMode(ledPin, OUTPUT);
```

```
}

void loop()
 {
  for (int brightness = 0; brightness <= 255; brightness++)
{
   analogWrite(ledPin, brightness);
   delay(10);
  }
  for (int brightness = 255; brightness >= 0; brightness--)
{
   analogWrite(ledPin, brightness);
   delay(10);
  }
}
```

The LED is connected to a PWM-enabled pin. The program gradually increases the PWM value from 0 to 255, causing the LED brightness to increase smoothly. It then decreases the value from 255 to 0, resulting in a smooth fade-out effect.

5. With a circuit diagram and code, explain how a push button connected to a digital output pin can be used to turn on/off an LED connected to a digital output pin.

A push button can be used as a digital input to control an LED connected to a digital output pin of an Arduino. When the button is pressed, the Arduino reads a HIGH signal and turns the LED ON. When the button is released, the input becomes LOW and the LED turns OFF.

6. What is meant by reading digital inputs in the 8051 microcontroller?

Reading digital inputs in the 8051 microcontroller refers to the process of detecting external logic levels applied to the I/O port pins. The 8051 has four 8-bit ports namely P0, P1, P2, and P3, which can be used as input ports. To read a digital input, the corresponding port pin must first be written with logic '1', which configures it as an input. When an external device such as a switch or sensor is connected, it drives the pin either to logic '0' or logic '1'. The microcontroller reads this logic level by accessing the port register. Proper pull-up or pull-down resistors are required to avoid floating inputs. Reading digital inputs is commonly used in applications like keypad interfacing, switch detection, and sensor monitoring.

7. What is the role of pull-up resistors while reading digital inputs?

The 8051 microcontroller includes internal pull-up resistors on Ports P1, P2, and P3, which help maintain a stable logic '1' when the port pins are used as inputs. These internal pull-ups are activated automatically when a logic '1' is written to the port pin. As a result, no external pull-up resistors are required for these ports, simplifying hardware design. When an external device pulls the pin low, the microcontroller reads logic '0', and when

the device is inactive, the internal pull-up keeps the pin at logic '1'. Port 0 is an exception, as it is an open-drain port and requires external pull-up resistors. Internal pull-ups reduce component count, cost, and circuit complexity.

Advantage of using internal pull-ups

- ❖ No need for external resistors
- ❖ Reduces hardware complexity
- ❖ Saves PCB space and cost

8. What is a pull-down resistor?

A pull-down resistor is a resistor connected between an input pin and ground to ensure that the pin remains at logic '0' when no external signal is applied. In the 8051 microcontroller, pull-down resistors are mainly used to prevent floating input conditions, which can cause unpredictable behavior. When a switch connected to the pin is open, the pull-down resistor forces the input to logic '0'. When the switch is closed, the input is driven to logic '1'. Pull-down resistors are especially important when using Port 0, since it does not have internal pull-up resistors. A typical value of a pull-down resistor is around 10 kΩ. They improve noise immunity and ensure reliable digital input readings.

9. Why is contact bounce a problem in digital systems?

Contact bounce is a common problem associated with mechanical switches, where the switch contacts do not settle immediately when pressed or released. Instead, the signal rapidly oscillates between logic '0' and '1' for a short duration, causing multiple false inputs. In the 8051 microcontroller, contact bounce can lead to incorrect readings, false triggering, or multiple counts for a single key press. To eliminate contact bounce, both hardware and software methods are used. Hardware debouncing uses RC circuits or Schmitt triggers to smooth the signal before it reaches the microcontroller. Software debouncing involves introducing a small delay and rechecking the switch status to confirm a valid press. Software debouncing is widely preferred due to its flexibility and ease of implementation.